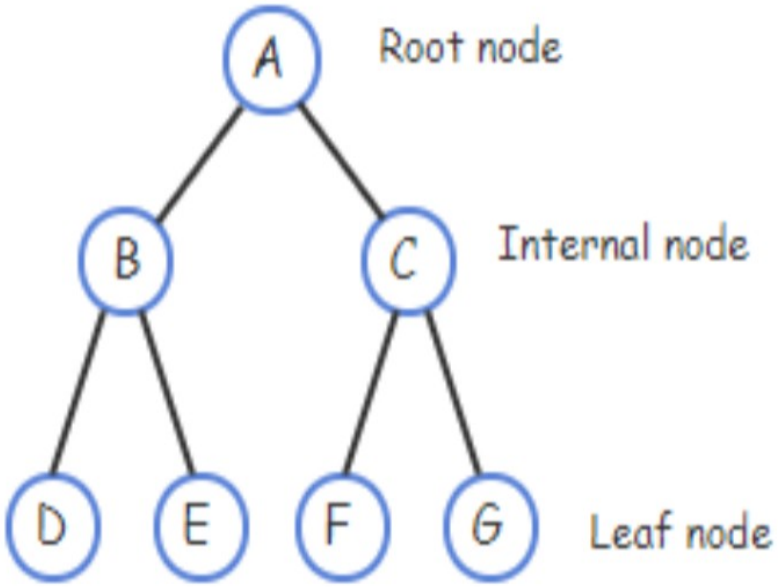
Introduction to Trees

Terminologies

Leaf: A tree node with no children.

Internal node: A node with at least one child.

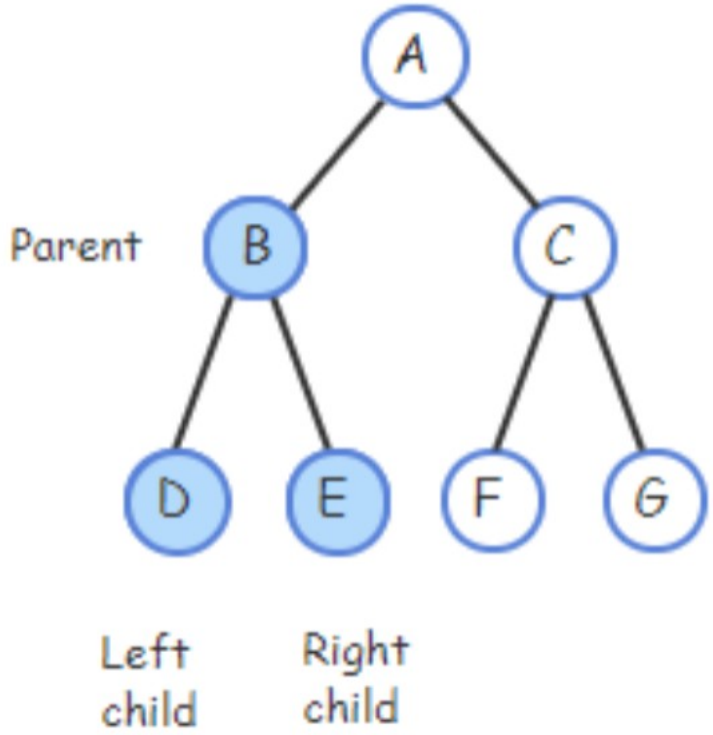
Root: The one tree node with no parent (the "top" node).



Terminologies

Parent: A node with a child is said to be that child's parent.

A node's ***ancestors*** include the node's parent, the parent's parent, etc., up to the tree's root.



Binary Tree Practice

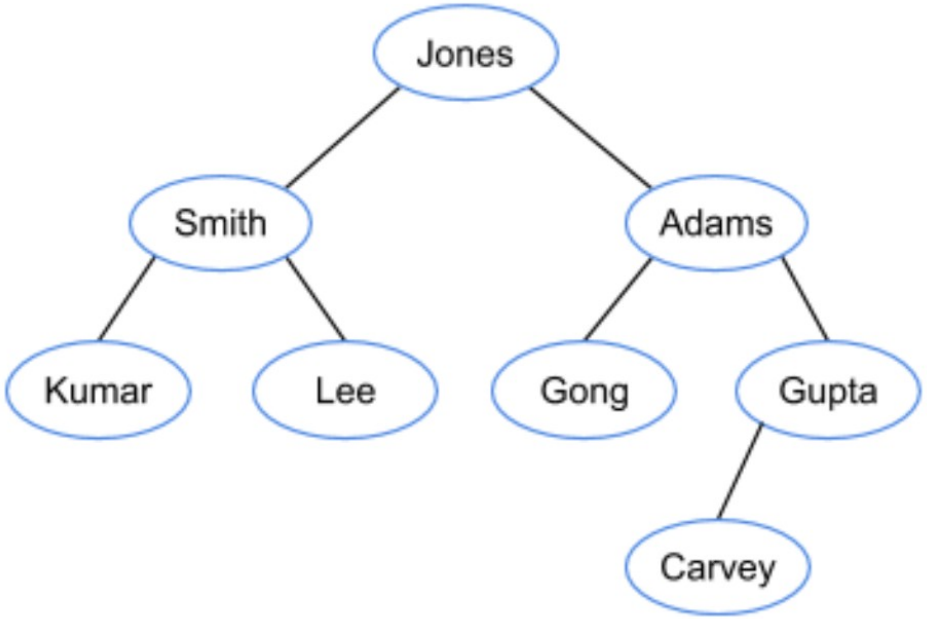
Root node:

Smith's left child:

Leaf nodes:

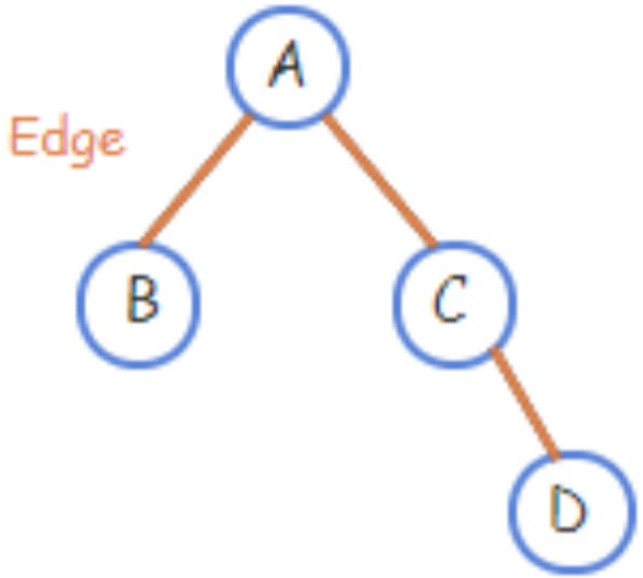
Number of internal nodes:

Gupta only has one child. Is this still a binary tree?



Edge

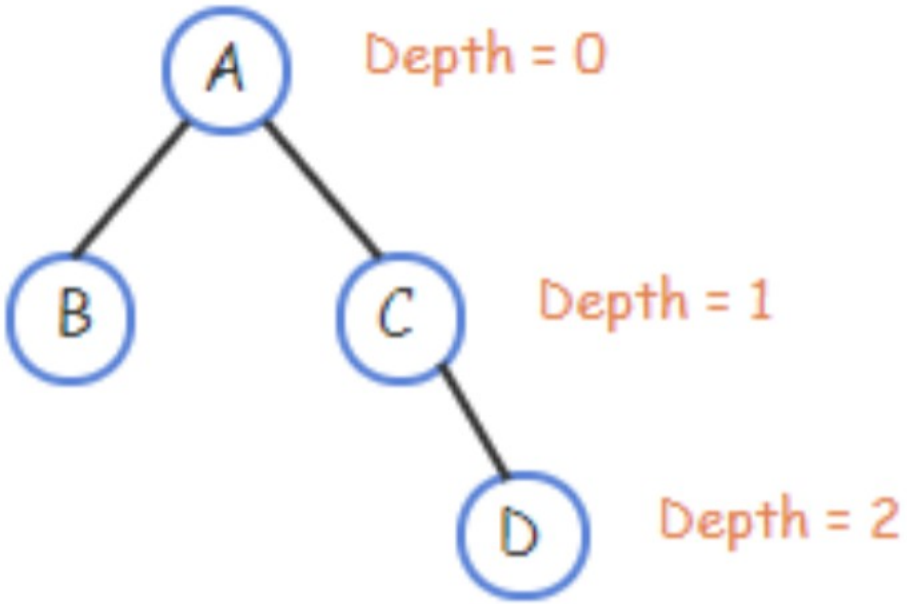
The link from a node to a child is called an ***edge***.



The binary tree has edges A to B, A to C, and C to D.

Depth

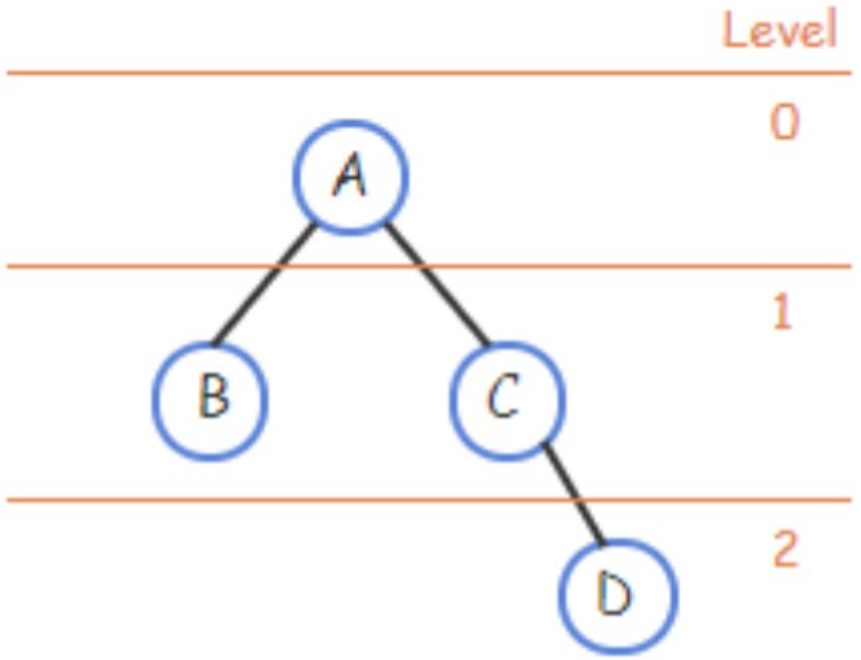
A node's ***depth*** is the number of edges on the path from the root to the node. The root node thus has depth 0.



A node's depth is the number of edges from the root to the node.

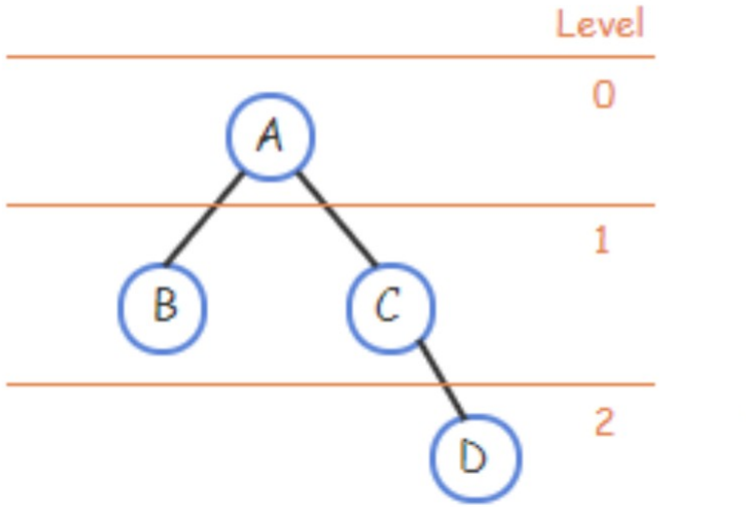
Level

All nodes with the same depth form a tree ***level***.



Height

A tree's ***height*** is the largest depth of any node. A tree with just one node has height 0.



Special types of binary trees

A binary tree is ***full*** if every node contains 0 or 2 children (***not full*** if there exists a node contains only 1 children).

A binary tree is ***complete*** if all levels, except possibly the last level, contain all possible nodes and all nodes in the last level are as far left as possible.

A binary tree is ***perfect***, if all internal nodes have 2 children and all leaf nodes are at the same level.

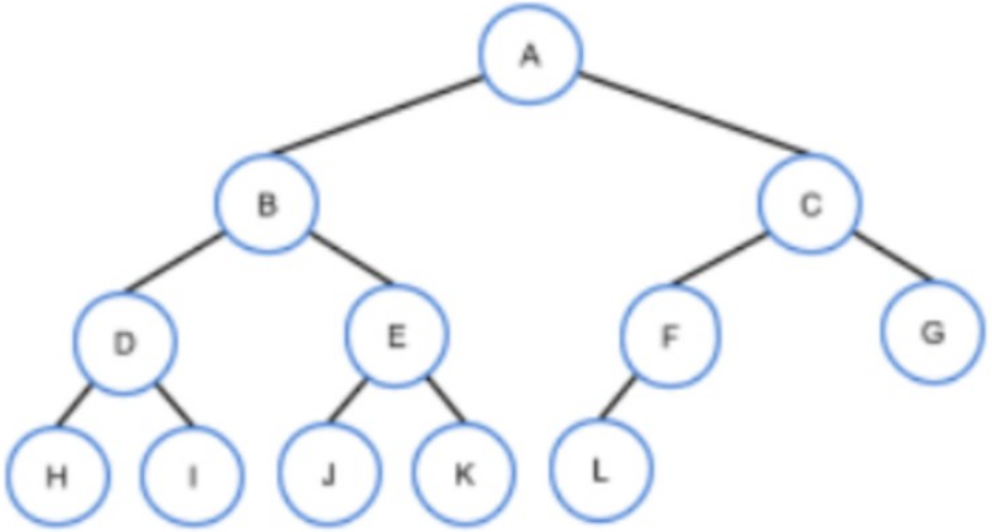
Special types of binary trees

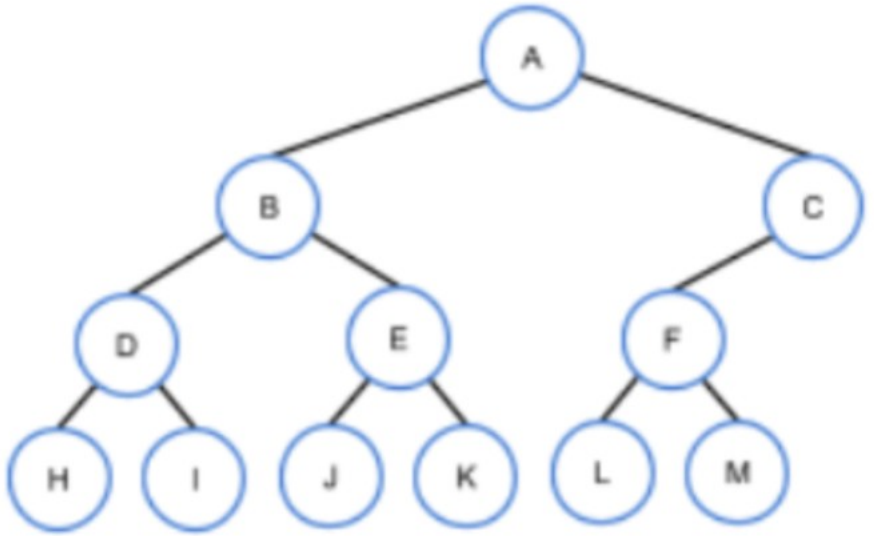
A binary tree is ***full*** if every node contains 0 or 2 children (***not full*** if there exists a node contains only 1 children).

A binary tree is ***complete*** if all levels, except possibly the last level, contain all possible nodes and all nodes in the last level are as far left as possible.

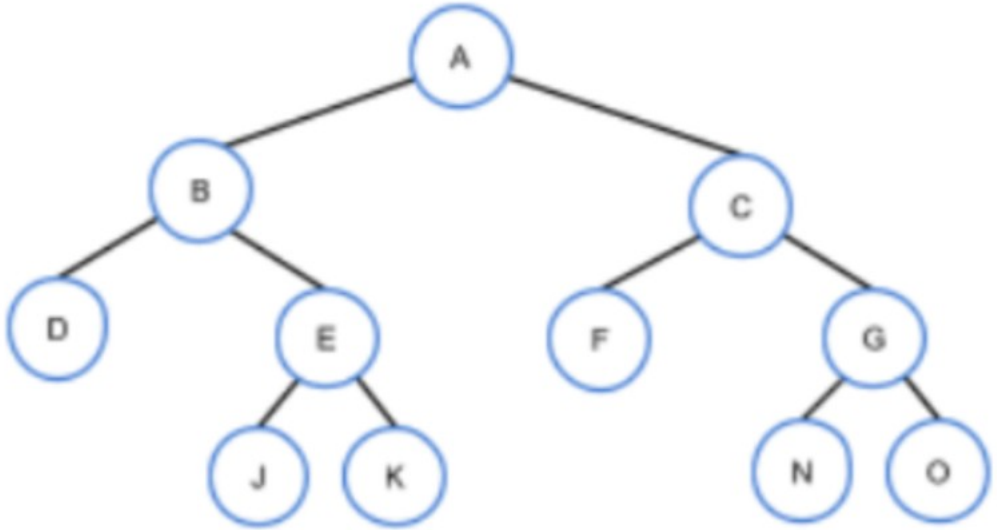
A binary tree is ***perfect***, if all internal nodes have 2 children and all leaf nodes are at the same level.

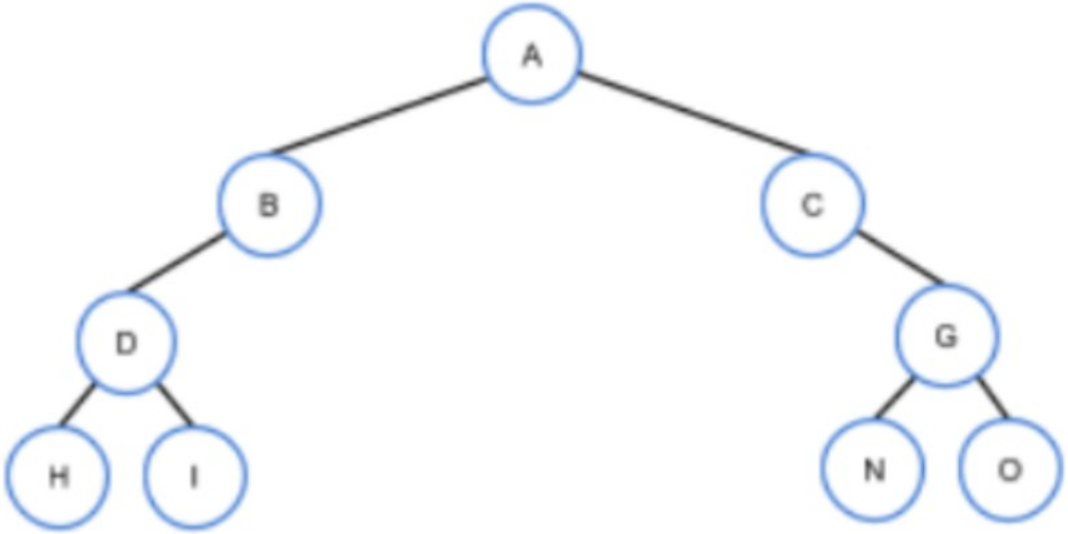
Special types of binary trees



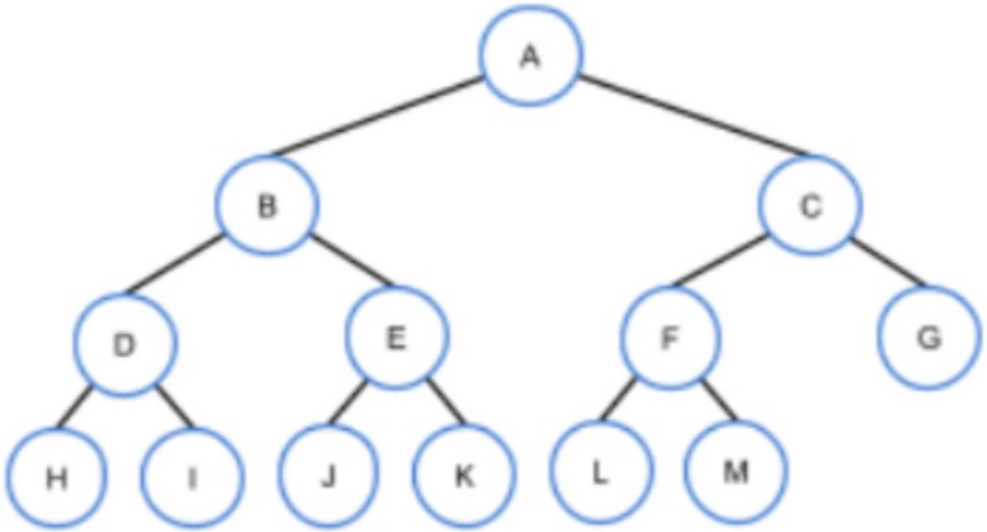


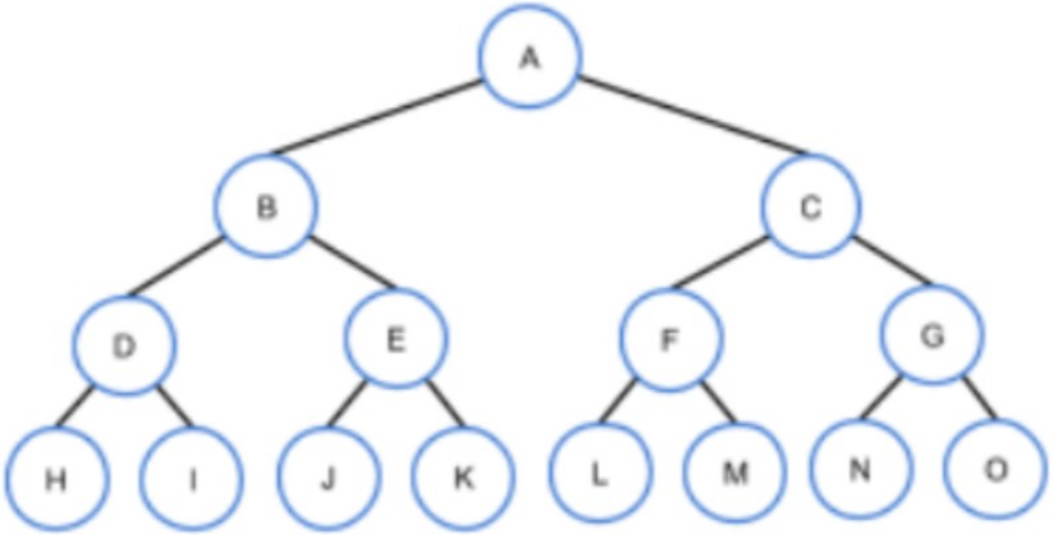
Special types of binary trees





Special types of binary trees





Binary Tree Traversal

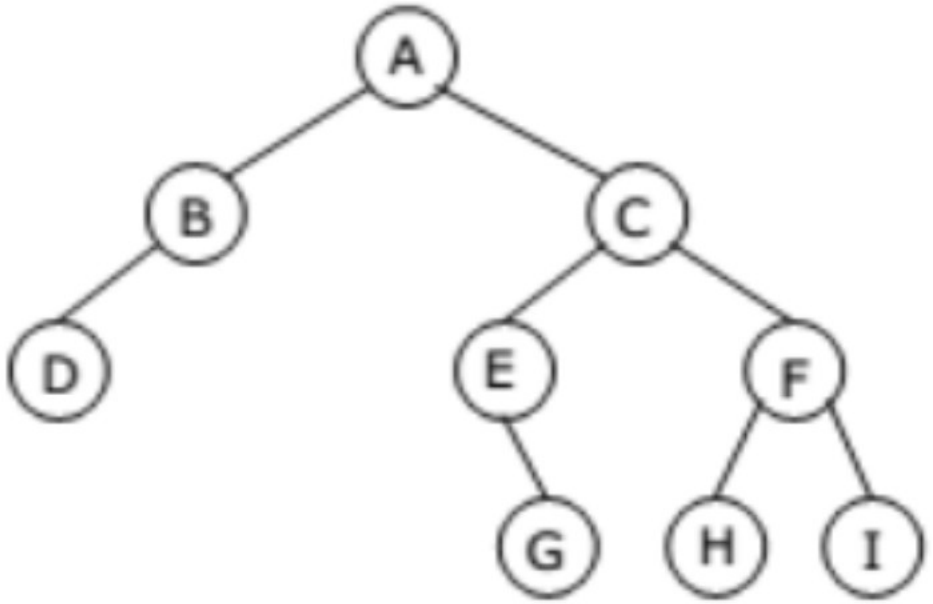
Depth First Traversals (go as deep as possible before go wider) :

- Inorder
- Preorder
- Postorder

Breadth First Traversals (go as wide as possible before go deeper):

- Level-order

In order Traversal



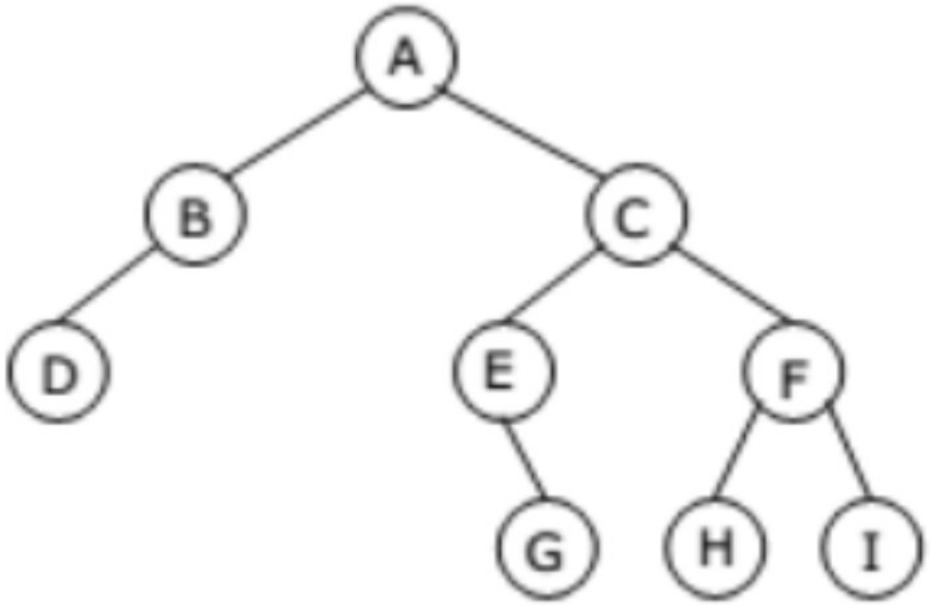
Visit the left subtree, using inorder

Visit the root

Visit the right subtree, using inorder

[D, B, A, E, G, H, F, I]

Pre order Traversal



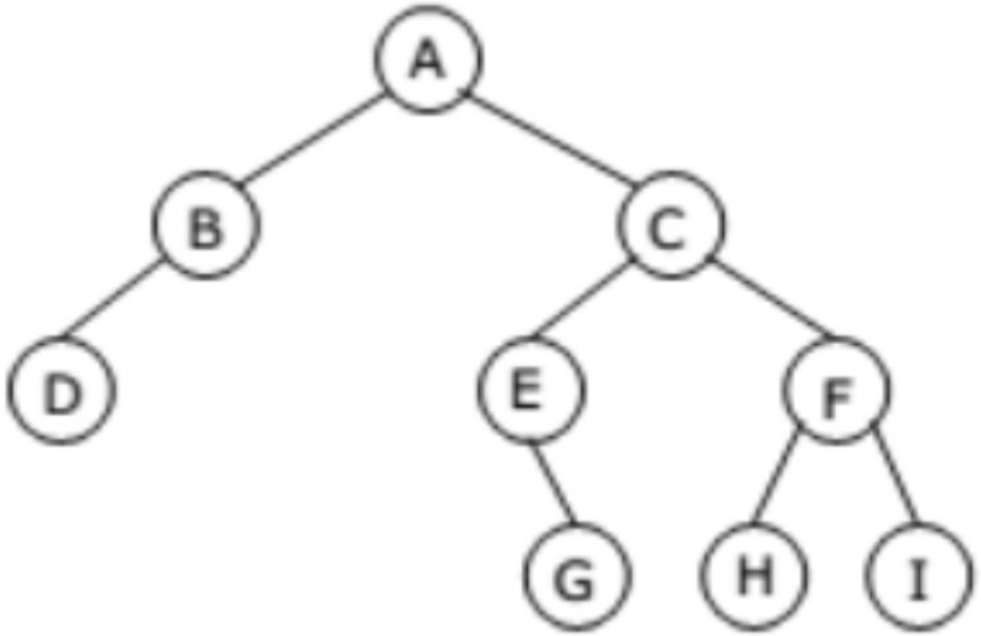
Visit the root

Visit the left subtree, using preorder

Visit the right subtree, using preorder

[A, B, D, C, E, G, F, H, I]

Post order Traversal



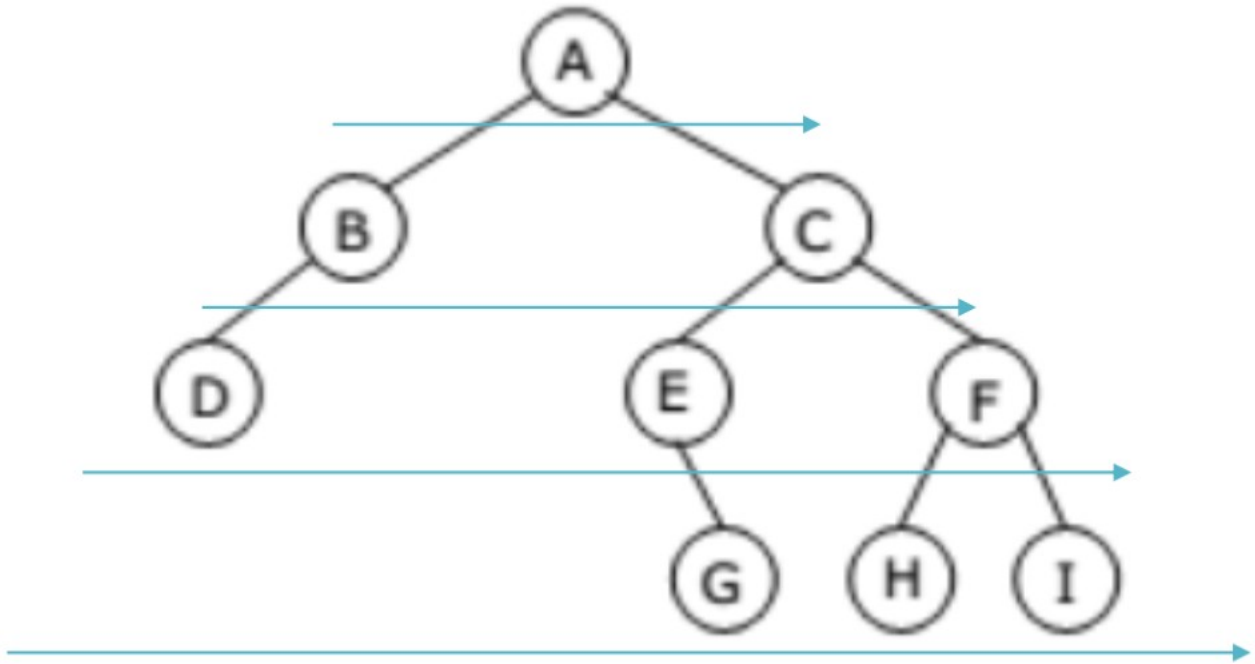
Visit the left subtree, using preorder

Visit the right subtree, using preorder

Visit the root

[D, B, G, E, H, I, F, C, A]

Level order Traversal



[A, B, C, D, E, F, G, H, I]

Depth First Level Pseudocodes

```
inorder(treeNode){
    if node is null:
        return;

    inorder(treeNode.left)
    print(treeNode.data)
    inorder(treeNode.right)
}
```

```
preorder(treeNode){
    if node is null:
        return;

    print(treeNode.data)
    preorder(treeNode.left)
    preorder(treeNode.right)
}
```

```
postorder(treeNode){
    if node is null:
        return;

    postorder(treeNode.left)
    postorder(treeNode.right)
    print(treeNode.data)
}
```